Python

Những chỗ Python khác C++

Chương 0 · Bài 0.1

Python trong khóa học

Từ bài sau, mọi code đều là Python: pandas, NumPy, scikit-learn, PyTorch.

Vòng lặp, hàm, thuật toán từ C++ vẫn dùng như cũ.

Bài này học những chỗ dễ làm sai:

- chia, lấy dư với số âm;
- hai biến dùng chung một list;
- notebook nhớ biến cũ.

Mục tiêu

- Tính đúng phép chia `//`, `%` với số âm; so sánh số thực.
- Biết khi nào hai biến dùng chung một list, và cách sao chép list.
- Chọn list, set hay dict theo độ phức tạp; đo thời gian chạy.
- Đọc, ghi file CSV; đọc thông báo lỗi để sửa code.

Thụt lề thay cho dấu ngoặc

```
def passed_scores(values, threshold=5.0):
    result = []
    for value in values:
        if value >= threshold:
            result.append(value)
    return result
```

- Dấu `:` mở một khối.
- Các dòng trong cùng khối thụt lề như nhau.
- `return` ngang hàng `for`: chạy sau khi lặp xong.
- `range(1, 5)` lấy 1, 2, 3, 4.

Notebook nhớ lần chạy gần nhất

1. Chạy cả hai ô

```
n = 3
```

```
total = sum(range(n + 1))
```

total = 6

2. Sửa n, chỉ chạy ô trên

```
n = 5
```

```
total = sum(range(n + 1))
```

total = 6 cũ

3. Chạy lại ô dưới

```
n = 5
```

```
total = sum(range(n + 1))
```

total = 15

Sửa n mà chưa chạy lại ô dưới thì total vẫn là 6.

Trước khi nộp: *Restart & Run all*.

Chạy trên Colab và Kaggle

- Colab: *Runtime → Change runtime type → T4 GPU*.
- Kaggle: *Session options → Accelerator*; cần xác minh số điện thoại.
- Chương 0–4 chạy được trên CPU; từ chương 5 nên bật GPU.

```
import torch

device = 'cuda' if torch.cuda.is_available() else 'cpu'
print('device:', device)
```

Bật GPU đúng thì in `cuda`.

HỎI NHANH

Trên Kaggle, mục *Accelerator* chỉ chọn được *None*. Lý do thường gặp nhất là gì?

A Notebook có lỗi cú pháp

B Tài khoản chưa xác minh số điện thoại

C Kaggle chỉ mở GPU vào cuối tuần

D Chưa cài `torch` bằng `pip`

HỎI NHANH

Notebook có hai ô: trên là `n = 3`, dưới là `total = sum(range(n + 1))`. Hôm qua đã chạy cả hai. Hôm nay mở lại (kernel mới), chạy ô dưới trước. Ô dưới cho ra gì?

A `total` bằng 0, vì `n` chưa gán thì coi là 0

B `total` bằng 6, vì Python đọc hết notebook trước

C Lỗi `NameError`, vì chưa có biến `n`

D `total` bằng 6, vì dùng lại `n` của hôm qua

Chia số âm: khác C++

```
print(7 / 2, 6 / 2)
print(7 // 2, -7 // 2)
print(7 % 3, -7 % 3)
```

KẾT QUẢ

```
3.5 3.0
3 -4
1 2
```

`//` làm tròn xuống. Với $b \neq 0$:

$$a = (a//b) \cdot b + (a\%b).$$

Số chia âm thì `%` không dương: `10 % -3` là -2.

`int` tăng độ chính xác theo nhu cầu; vẫn tốn thời gian và bộ nhớ khi số rất lớn.

Làm tròn xuống, không phải về 0



// làm tròn xuống; phép chia nguyên của C++ cắt về 0.

HỎI NHANH

`print(-7 // 2, -7 % 2)` in ra gì?

A `-3 -1`

B `-4 1`

C `-3 1`

D `-3.5 1`

Số thực cần kiểm sai số

```
import math

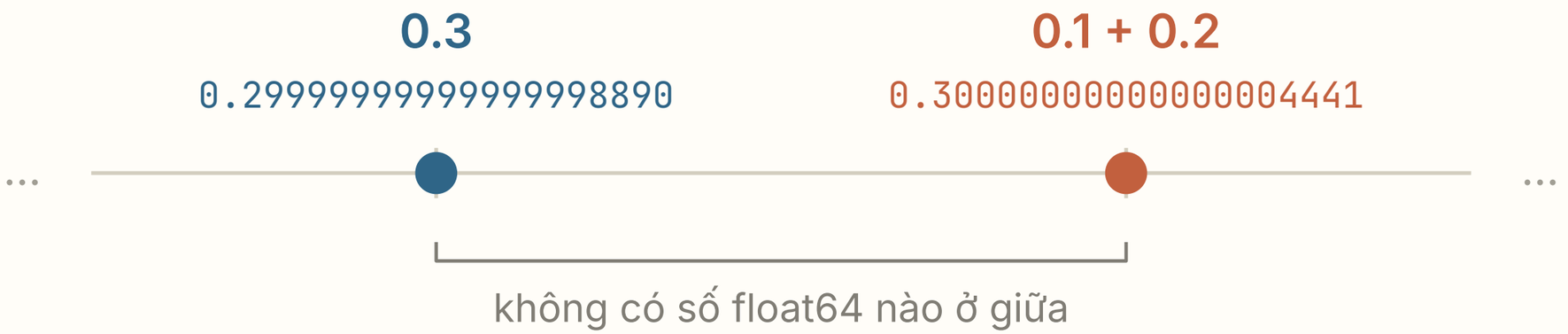
print(0.1 + 0.2, 0.1 + 0.2 == 0.3)
print(math.isclose(0.1 + 0.2, 0.3,
                   rel_tol=1e-9, abs_tol=1e-12))
print(round(2.5), round(3.5), round(0.125, 2))
```

KẾT QUẢ

```
0.30000000000000004 False
True
2 4 0.12
```

- 0.1 không lưu chính xác được trong nhị phân.
- So số thực bằng `math.isclose`, không dùng ==.
- `round` chọn mốc chẵn khi số nằm đúng giữa.

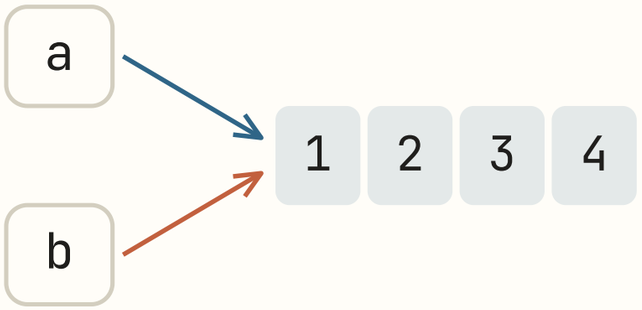
0.1 + 0.2 và 0.3 là hai số máy khác nhau



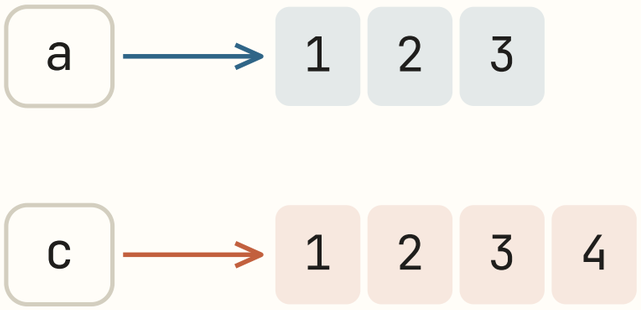
0.3 và 0.1 + 0.2 rơi vào hai số float64 liền kề.

Gán tên khác với sao chép

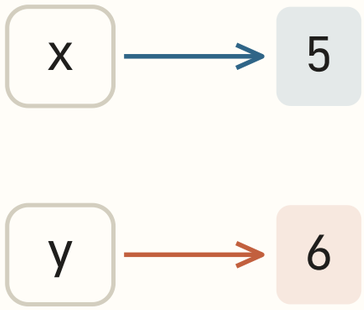
```
b = a
b.append(4)
```




```
c = a.copy()
c.append(4)
```




```
y = x
y += 1
```



Gán chỉ dán thêm tên; copy() tạo list mới; int bất biến.

Sửa list, hay gán lại tên?

```
a = [1, 2, 3]
b = a
b.append(4)
print(a, b, a is b)
```

KẾT QUẢ

```
[1, 2, 3, 4] [1, 2, 3, 4] True
```

- `append`, `a += [5]`: sửa list chung.
- `a = a + [5]`: tạo list mới, gán lại tên `a`.
- `is`: cùng đối tượng? `==`: giá trị bằng nhau?

HỎI NHANH

```
def reset(scores):
    scores.append(0)
    scores = []
marks = [7, 8]
reset(marks)
print(marks)
```

Đoạn code in ra gì?

A `[]`

B `[7, 8]`

C `[0]`

D `[7, 8, 0]`

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 94

List lồng nhau: một hàng hay ba hàng?

```
grid = [[0] * 3] * 3
grid[0][0] = 9
print(grid)

grid = [[0] * 3 for _ in range(3)]
grid[0][0] = 9
print(grid)
```

KẾT QUẢ

```
[[9, 0, 0], [9, 0, 0], [9, 0, 0]]
[[9, 0, 0], [0, 0, 0], [0, 0, 0]]
```

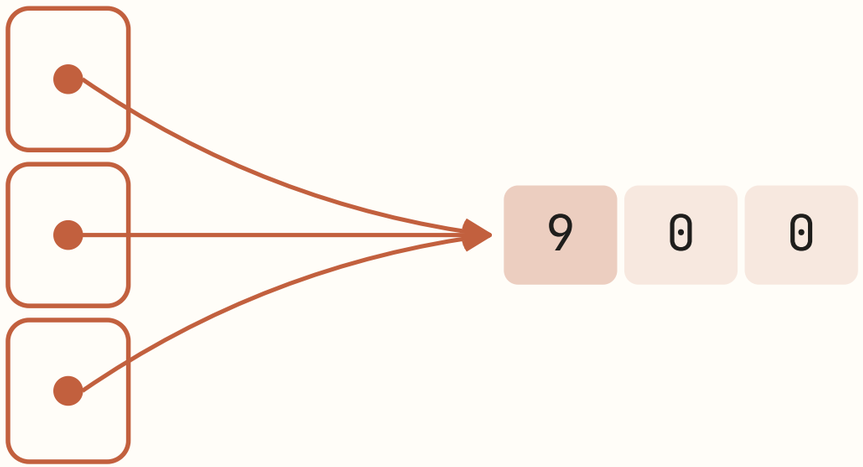
`* 3` lặp lại tên của cùng một hàng, không tạo hàng mới.

Comprehension tạo một hàng mới ở mỗi lượt lặp.

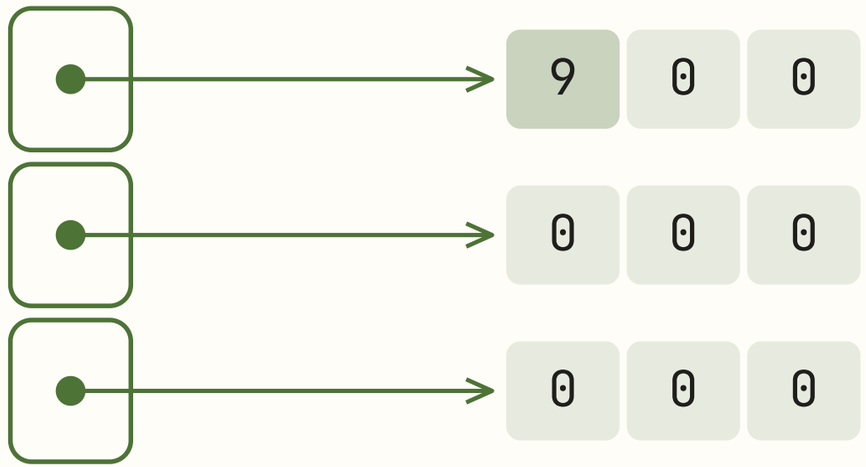
`copy()` chỉ chép một tầng; `deepcopy()` chép mọi tầng.

Ba tên, một hàng

```
[[0] * 3] * 3
```




```
[[0] * 3 for _ in range(3)]
```



Gán 9 vào ô đầu: bên trái cả ba hàng đổi, bên phải chỉ một hàng đổi.

HỎI NHANH

Chạy `a = [[1, 2], [3, 4]]`, `b = a.copy()`, `b[0].append(9)`. Lúc này `a` là gì?

A `[[1, 2], [3, 4]]`

B `[[1, 2, 9], [3, 4]]`

C `[[1, 2], [3, 4], 9]`

D Báo lỗi

Chọn cấu trúc như khi thi lập trình

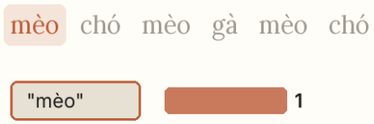
Công việc	Cấu trúc
Giữ thứ tự, lấy bằng vị trí	`list`
Một bộ giá trị cố định	`tuple`
Tra cứu theo khóa	`dict`
Kiểm thuộc tập nhiều lần	`set`

`x in list`: $O(n)$ ở trường hợp xấu.

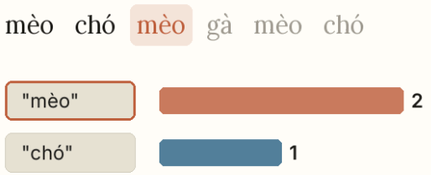
`x in set`: trung bình $O(1)$, nhưng dựng set cũng tốn $O(n)$.

Khóa của dict, phần tử của set phải **hashable**. `(3, 4)` được; list, hay tuple chứa list, thì không.

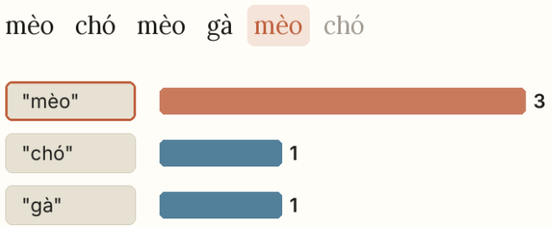
Đếm bằng dict



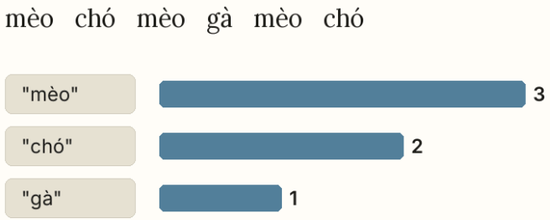
từ “mèo”: khóa mới, đặt bằng 1



từ “mèo”: đã có khóa, tăng 1 → 2



từ “mèo”: đã có khóa, tăng 2 → 3

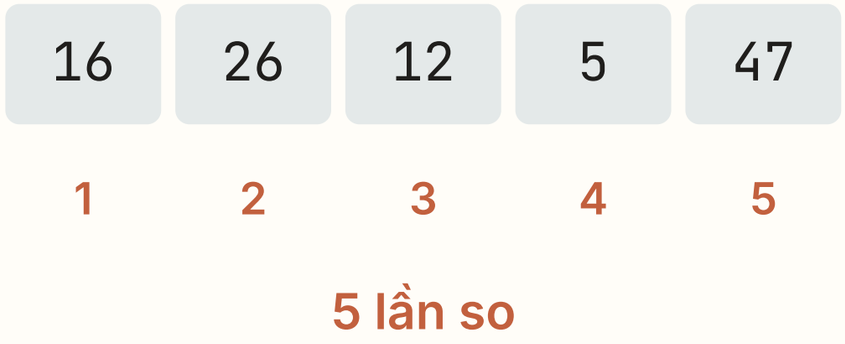


`most_common(2)` sắp theo số lần giảm dần: mèo 3, chó 2

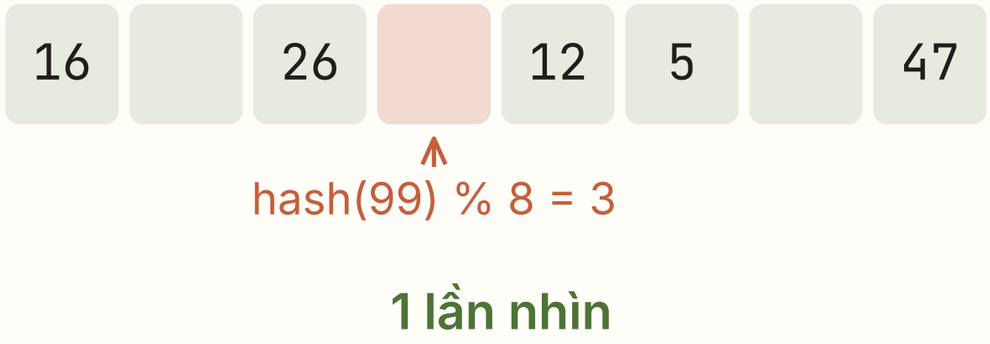
Đếm “mèo chó mèo gà mèo chó” bằng dict: mỗi từ chỉ chạm đúng một khóa, khóa mới bắt đầu từ 1.

Vì sao `in` trên set nhanh

`99 in list`



`99 in set`



List so từng phần tử; set nhảy thẳng tới một ngăn của bảng băm.

HỎI NHANH

Có 50 000 mã đã nộp bài. Cần kiểm 10 000 mã khác xem đã nộp chưa. Nên làm thế nào?

A Đổi danh sách thành `set` rồi dùng `in`

B Cứ dùng `in` trên list, 50 000 chưa nhiều

C Sắp xếp list trước cho `in` nhanh hơn

D Viết hai vòng `for` lồng nhau

Slicing, comprehension và vài hàm quen tay

```
a = [5, 3, 8, 1, 9, 2]
print(a[1:4], a[-1], a[-2:], a[::-1])
print([x * x for x in a if x % 2 == 1])
```

KẾT QUẢ

```
[3, 8, 1] 2 [9, 2] [2, 9, 1, 8, 3, 5]
[25, 9, 1, 81]
```

- `enumerate`: vị trí và phần tử.
- `zip`: phần tử cùng vị trí của nhiều dãy.
- `sorted(..., key=...)`: sắp theo một giá trị.
- `d.get(key, default)`: xử lý khóa chưa có.

Tham số mặc định được tạo một lần

```
def add_score(x, history=[]):
    history.append(x)
    return history

print(add_score(1), add_score(2))
```

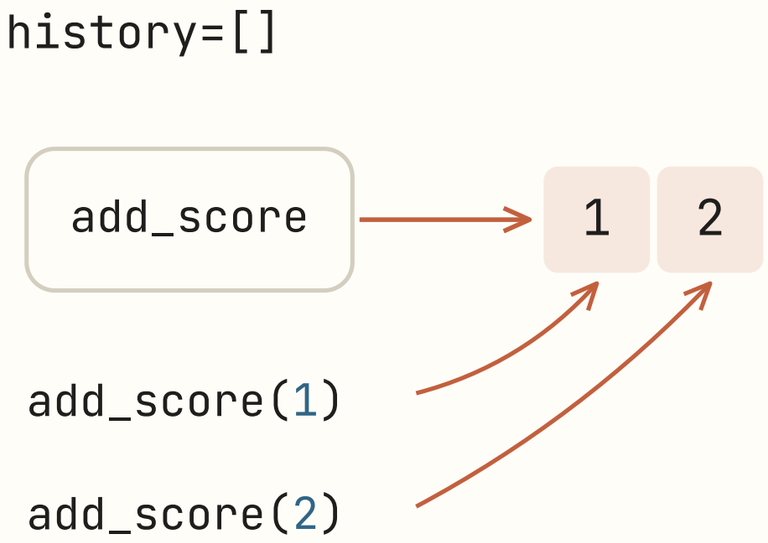
KẾT QUẢ

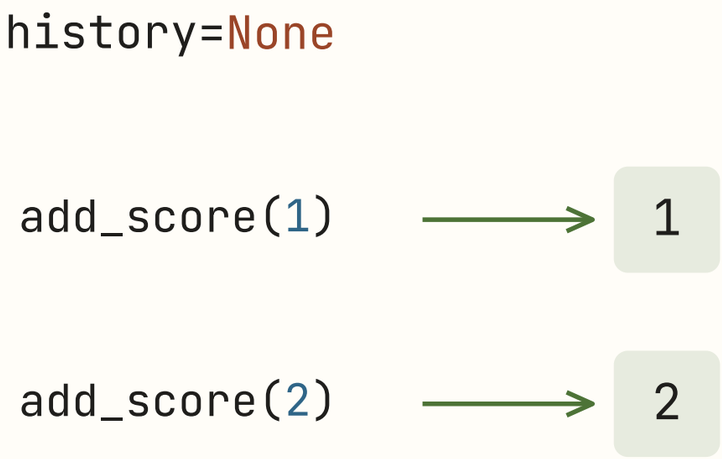
```
[1, 2] [1, 2]
```

Hai lần gọi dùng chung list mặc định.

```
def add_score(x, history=None):
    if history is None:
        history = []
    history.append(x)
    return history
```

Một list mặc định cho mọi lần gọi





List mặc định tạo một lần và dùng chung; None cho mỗi lần gọi một list mới.

HỎI NHANH

`add_score(score, history=[])` gọi lần sau vẫn thấy điểm của lần trước. Sửa thế nào để mỗi lần gọi bắt đầu từ list rỗng?

A Giữ `history=[]`, gọi `clear()` ở cuối hàm

B Giữ nguyên, vì Python tạo lại list mỗi lần

C Đặt `history=None`, tạo list mới trong hàm

D Dùng list toàn cục, gán `[]` trước mỗi lần

Kiểm đáp án trước khi đo

Tổng $0 + 1 + \cdots + (n-1)$:

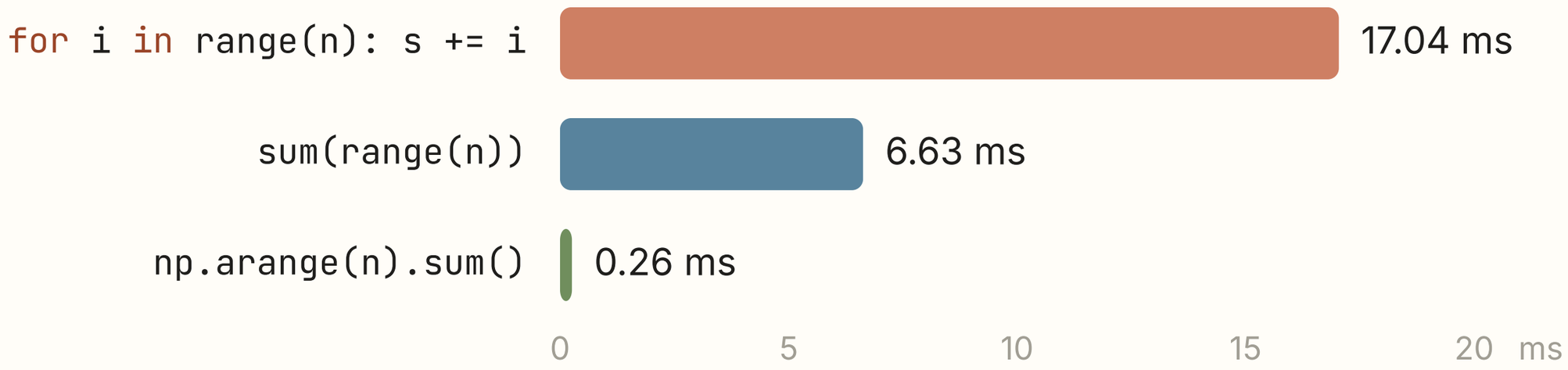
$$S = \frac{n(n-1)}{2}.$$

Với $n = 500\,000$: **124 999 750 000**.

Ba cách: vòng `for`, `sum(range(n))`, NumPy. Cả ba phải ra đúng số này.

Rồi mới đo bằng `timeit.repeat`, mỗi cách chạy nhiều lần.

Vòng `for` chậm nhất



Cùng một tổng của 500 000 số, đo trên một máy CPU.

`sum` và NumPy lặp bên trong code C nên nhanh hơn. Công thức $n(n-1)/2$ còn nhanh hơn cả ba.

HỎI NHANH

Hai cách đều tính đúng. Cách A chạy 8 ms nhưng phải chuẩn bị dữ liệu thêm 5 ms. Cách B chạy 11 ms, không cần chuẩn bị. Chỉ chạy một lần thì cách nào nhanh hơn?

A B, vì 11 ms ít hơn 13 ms

B Như nhau, vì cùng độ phức tạp

C Chưa so được, vì số đo dao động

D A, vì 8 ms ít hơn 11 ms

Ghi CSV rồi đọc lại

`preds` là dict ID → dự đoán.

```
with path.open(
    "w", newline="", encoding="utf-8"
) as file:
    writer = csv.writer(file)
    writer.writerow(["id", "prediction"])
    for student_id, prediction in preds.items():
        writer.writerow(
            [student_id, prediction]
        )
```

- `with` đóng file khi ra khỏi khối.
- Ghi rõ `encoding="utf-8"`.
- Đọc lại bằng `csv.reader`: mọi ô là chuỗi, đổi kiểu trước khi tính.

HỎI NHANH

Đọc cột điểm bằng `csv`, ta được `["9", "10", "2"]`. `sorted` trả về gì?

A `['2', '9', '10']`

B `['10', '2', '9']`

C `['9', '10', '2']`

D `TypeError`

Đọc traceback từ dưới lên

```
Traceback (most recent call last):
  File "…/diem.py", line 7, in <module>
    print(mean_of("Dũng"))
          ~~~~~~~^^^^^^^^
  File "…/diem.py", line 4, in mean_of
    marks = scores[name]
            ~~~~~~^^^^^^
KeyError: 'Dũng'
```

③ ai đã gọi

② ở dòng nào

① lỗi gì

đọc từ dưới lên

Traceback thật của Python 3.13: đọc từ dòng cuối lên.

Bốn lỗi hay gặp

Bốn thông báo cuối của bốn traceback riêng:

KẾT QUẢ

```
TypeError: can only concatenate str (not "int") to str
KeyError: 'Dũng'
IndexError: list index out of range
ValueError: invalid literal for int() with base 10: '7.5'
```

- Dòng cuối: loại lỗi và nguyên nhân.
- Các dòng `File ..., line ...`: vị trí cần mở code.
- Không báo lỗi: kiểm kiểu, độ dài, vài giá trị.

HỎI NHANH

`marks` có 3 phần tử. Vòng `for i in range(1, len(marks) + 1): print(marks[i])` chạy ra sao?

A In đủ ba phần tử như vòng 1 đến n của C++

B Báo `IndexError` ngay ở vòng đầu

C In `marks[1]`, `marks[2]` rồi dừng

D In `marks[1]`, `marks[2]`, rồi `IndexError`

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 137

Class: dữ liệu đi cùng hàm

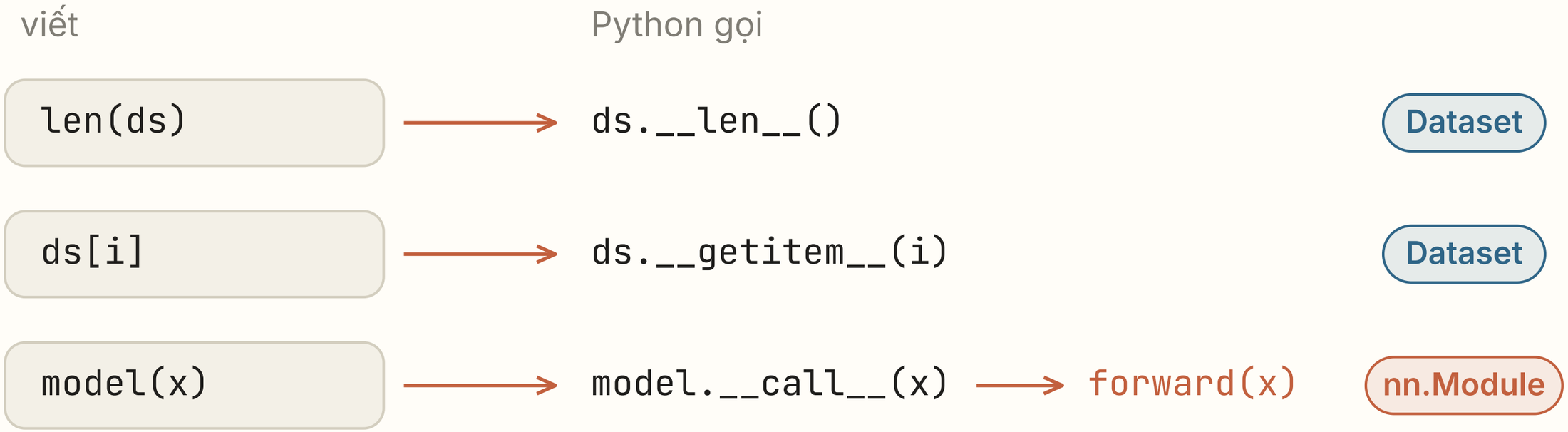
```
class ScoreBook:
    def __init__(self, names, marks):
        self.names = list(names)
        self.marks = list(marks)

    def __len__(self):
        return len(self.marks)

    def __getitem__(self, i):
        return self.names[i], self.marks[i]
```

- `__init__` chạy khi tạo đối tượng; `self` là chính đối tượng.
- `len(book)` gọi `__len__`, `book[1]` gọi `__getitem__`.
- Class con gọi `super().__init__(...)` để chạy phần khởi tạo của cha.
- `Dataset` và `nn.Module` ở chương 3 dùng đúng các hàm này.

Cú pháp và hàm đặc biệt



Ba cú pháp và hàm đặc biệt mà chúng gọi.

HỎI NHANH

Xóa dòng `super().__init__(factor)` trong `ShiftScaler.__init__`, rồi gọi `ShiftScaler(2, 1)([1, 2, 3])`. Điều gì xảy ra?

A Vẫn ra `[3, 5, 7]`, vì Python tự gọi `__init__` của class cha

B Ra `[2, 3, 4]`, vì thiếu `factor` thì Python coi là 1

C Báo `AttributeError`, vì đối tượng chưa có thuộc tính `factor`

D Báo lỗi ngay ở dòng tạo đối tượng, trước khi gọi

Khi đi thi

- *Restart & Run all* trước khi nộp.
- Nhiều số thì dùng `sum` hoặc NumPy.
- `encoding="utf-8"`, đọc lại file nộp.

02

Luyện đề

LUYỆN ĐỀ 1

Sau `a = [1, 2]`, `b = [1, 2]`, `c = a`, lệnh `print(a == b, a is b, a is c)` in ra gì?

A `True True True`

B `True False True`

C `False False True`

D `True False False`

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 95

LUYỆN ĐỀ 2

Quen C++, một bạn viết `[1, 2] + [3, 4]` để cộng từng phần tử. Python cho kết quả gì?

A `[4, 6]`

B `[[1, 2], [3, 4]]`

C Báo lỗi

D `[1, 2, 3, 4]`

ĐỀ THI NOAI 2026 Singapore · Vòng sơ loại · Câu 91

LUYỆN ĐỀ 3

Có `words = ["mèo", "chó", "mèo"]`. Cần gán cho mỗi từ khác nhau một số 0, 1,... Chạy lại lần nào cũng phải ra đúng một bảng. Viết thế nào?

A
```
{w: i for i, w in enumerate(sorted(set(words)))}
```

B
```
{w: i for i, w in enumerate(list(set(words)))}
```

C
```
{w: i for i, w in enumerate(sorted(words))}
```

D
```
{w: words.index(w) for w in words}
```

PHỎNG THEO USA-NA-AIO 2026 · Vòng 1 · Part 5.1–5.2

Chốt ý

`//`, `%` làm tròn xuống, khác C++ với số âm.

Gán không sao chép; cần list riêng thì `copy()`.

Tính trên nhiều số thì dùng `sum` hoặc NumPy.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook: *Restart & Run all*, rồi thử đổi `n` mà không chạy lại ô dưới.